Do you find hackers cool?

The Journey of Your First Terminal Command — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

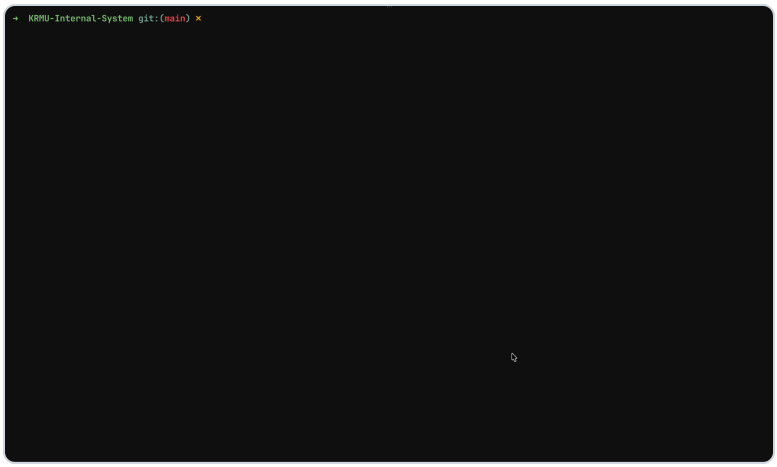
How to use this sheet: Keep your Terminal open beside you. For every task, **attempt and write your answer *before* we discuss it** — your first guesses do not need to be right. Thinking hard and being wrong is exactly how this is meant to work.

Pre-requisites → For users on Windows, install Windows Subsystem for Linux (WSL).

Block A — What is it exactly?

Q1. Suppose you have to create a separate folder for every student in this class, with each folder named after their roll number. How would you do it? What do you think is the **most efficient approach**?

Q2. You've probably seen hackers, programmers, or cybersecurity experts in movies using a screen that looks something like the one below. What do you think they are using? What do you think this application is **called**, and what do you think they are doing inside it?



(for fun — search hackertyper.com and make yourself look cool)

Block B — Writing your first terminal commands

TRY THIS · TERMINAL

Open your Terminal on the left half, and file explorer/finder on the right half, and run `pwd`. You must have observed something similar to this:

```
➤ nikk pwd
/users/nikk
➤ nikk
```

Q3. Using the output above, answer:

(a) What does the output represent, and what is the GUI equivalent for this?

(b) What does `pwd` stand for?

Q4. How can we check the contents of our current directory using the command line? (Can we **list** all the files?)

Now create a folder in the directory with your name using the file explorer / GUI.

Q5. How do you create a new directory using the terminal? (Can you **make a directory** in this folder?)

Q6. After running the command, how can you verify the new directory was created using the terminal? (Hint: **list** all files again.)

You can easily enter a folder through the GUI by double-clicking it.

Q7. Can you enter a folder through the terminal? (Hint: **change the directory**.)

Create a new file called `hello.txt` through the GUI and write "hello world" in it.

Q8. Can we create our files through the terminal? (How to **touch** files in our directory?)

Q9. Why do you think the command is called so? What could be the significance? (There's no right or wrong.)

Q10. Try reading the file through the terminal. What command can help you do so?

Block C — Editing Files from the Terminal

So far we have used terminal commands to create a directory, change directory, list all files, and create new files.

TRY THIS · TERMINAL

Can we write to files using the terminal? Try creating a file `hello.txt` through the terminal and type `vi hello.txt`.

Q11. What do you think happened? Are you stuck inside and not able to exit this?

Q12. What are the different "modes" in Vim? Try pressing `i` on your keyboard. What changed at the bottom of the screen?

Q13. Now that you are in the right mode, type a simple message like "Hello World". How do you tell Vim you are done typing?

Q14. Finally, how do you save your work and exit?

Congratulations! You've just edited, saved, and exited a file using Vim — the very tool that scares most budding programmers. You're officially on your way, and just completed the hardest part. Trust us, the rest is easy..

Block D — The Art of Digital Manipulation

Files and folders aren't just icons; they are data waiting to be molded. Let's see if you can bend them to your will through these terminal experiments:

Q15. You've wandered too deep into your directory maze. How do you step back to where you came from?

Q16. We often need to duplicate an important file instantly. How do you create a **clone**?

Q17. What if your file needs a new home or a new name? How do you move or rename it?

Q18. What if you want to banish a file into the void? How do you delete it forever? Is this **risky**?

Block E — Peeking Under the Hood

So far, you've been typing commands, but what's actually happening? Let's clear up the terms:

- **Terminal:** This is the window or application you are currently using. It's just the interface that lets you talk to your computer.
- **Shell:** This acts as the bridge that listens to what you type in the terminal. It takes your text commands and translates them so the computer knows what to do.
- **Bash:** There are many different types of shells out there, and Bash is the most common one. It's simply a specific program that acts as that bridge between you and the computer.

Questions:

Q19. Now that you've used them, how would you describe the difference between the **Terminal** and the **Shell** in your own words?

Q20. Why do you think there are different types of shells like Bash, instead of just one standard one for everyone?

Q21. Think about the relationship between the Terminal and the Shell: if the Terminal is the "window" or the interface you see, what role does the Shell play in actually getting things done?

Q22. Bash is just one of many different shells out there. If you were to swap it out for another shell (like **Zsh** or **Fish**), do you think the commands you've learned today would still work, or would everything break? Why?

Block F — Installing Git Bash

>> Download the installer from the official Git for Windows website (gitforwindows.org).

>> Run the downloaded installer file.

>> Follow the installation wizard, keeping the default settings for most options.

>> Click "Finish" to complete the installation.

>> Open your Terminal or Git Bash application to verify the installation.